

RE (concatenation, or, star Σ (Sigma), ϵ (epsilon), union, intersection, minus) RE can be used to specify keywords (reserved word) and identifiers (user-defined name).

Language is any subset of the set of all strings in a given alphabet.

The empty set is a language.

An empty string (a string of length 0, which is an element, not a set) is not a language.

A language $\{\epsilon\}$ is not empty; it is a set containing only one element ϵ , therefore it is not an empty set; its cardinality $|\{\epsilon\}| = 1$.

$A\emptyset = \emptyset A = \emptyset$

$\text{Lang}(R_1 \mid R_2) = \text{Lang}(R_1) \cup \text{Lang}(R_2)$

$A\{\epsilon\} = \{\epsilon\}A = A$

$(a|b)^* = \{a, b\}^*$

$(0|1)^3$ means selecting 3 times independently from the set $\{0, 1\}$ and concatenating the results in order.

Automata:

DFA

1. **Q**: a finite set of states 2. **Σ** : input alphabet 3. **δ** : $Q \times \Sigma \rightarrow Q$ (delta), transition function ($\delta(q, a) = q'$) 4. **q0** $\in Q$ is the start state 5. **F** $\subseteq Q$: set of final states.

Complement: swap final and non-final states

Acceptance: if w is in $\text{Lang}(D)$

non-emptiness problem: return Yes if $L(M) \neq \emptyset$ and No otherwise. Use a graph-traversal algorithm (adjacency-list representation, DFS in time $O(n+m)$), starting at the initial state, to see if there is a final state that is reachable from the initial state. (**linear equivalence problem** (polynomial in the sizes of the inputs M_1, M_2): return Yes if $L(M_1) = L(M_2)$, and No otherwise. $L_1 \subseteq L_2$ and $L_2 \subseteq L_1$.

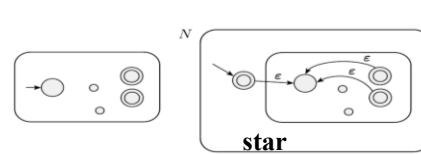
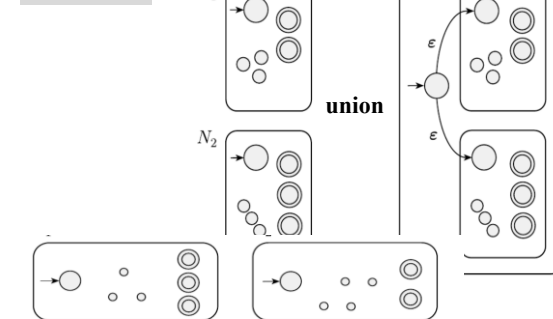
$L_1 \cap (\Sigma^* \setminus L_2) = \emptyset \rightarrow L_1 \subseteq L_2$ 1. For $i = 1, 2$, build DFAs M'_i such that $L(M'_i) = \Sigma^* \setminus L(M_i)$.

2. Form DFAs J for $L(M_1) \cap L(M'_2)$, and K for $L(M_2) \cap L(M'_1)$. 3. Test if both $L(J) = \emptyset$ and $L(K) = \emptyset$. If so, return Yes, else return No.

A **decision problem** is a computational problem

DFA

RE to NFA:



A language $L \subseteq \Sigma^*$ is called **regular** if $L = \text{Lang}(D)$ for some DFA D .

Not regular (view as string): Arithmetic expressions, Boolean expressions, Regular expressions, Python programs. arbitrarily deep, nested structure: $()$, blocks

For every DFA D there is a string x that **witnesses** that $L(D) \neq L$: either $x \in L(D) \setminus L$ or $x \in L \setminus L(D)$

Eg: $\{a^i b^j : i > j\}$ If there were a DFA for it, then there exists $i > j$ such that a^i, a^j go to the same state. But then if we extend by b^j , then $a^i b^j \in L$ while $a^j b^j \notin L$, which is impossible.

Proof of a negation: 1. Assume L_1 is regular. 2. Express L_2 (not regular) = L_1 (using union, intersection, complementation, cat, and star) regular. 3. This is an impossible situation (the contradiction). 4. Conclude L_1 is not regular

Closed under an operation if: Whenever you apply the operation to elements of the set, the result is still an element of the same set.

The regular languages are closed under the Boolean operations union, intersection, complementation, as well as concatenation and Kleene star.

L is regular, then $OP(L)$ is regular: Build NFA

a: +2

RS: +0 (ϵ -link R.accept $\rightarrow$ S.start)

R|S: +1 (new start, $\epsilon \rightarrow$ both)

R*: +1 (new start=accept, $\epsilon \rightarrow$ R.start, R.accept $\epsilon \rightarrow$ start)

DFA from NFA: **no ϵ -transitions ever**

NFAs to NFAs without ϵ -transitions:

1. If $EC(q, r)$ and $\delta(r, a) = s$ then add $\delta(q, a) = s$ to M . 2. If $EC(q, r)$ and r is final, make q final in M ps. $EC =$ Epsilon Closure, $EC(q, r) = q \rightsquigarrow r$

NFAs without ϵ -transitions to DFAs:

subset construction: set of states in DFA ($2^{|Q|}$)

Then put all q in $\delta(q$ in Q' , input) into $\delta'(Q', \text{input})$

Just from 0,1,2 then expand, not write all subsets

DFAs to Regular Expressions:

GNFAs (Generalised):

1) the initial state has no incoming edges. 2) there is one final state, and it has no outgoing edges. 3) there are edges from every state (that is not final) to every state (that is not initial).

1. From DFAs to GNFAs: 1) Add an initial state q_s and ϵ -transition to the old initial state. 2) Add a final state q_f and ϵ -transitions to it from all the old final states. 3) Add transitions for every pair of states, including from a state to itself, (except leaving the final, or entering the initial). (add $\delta(q, \emptyset) = r$)

2. Removing states from GNFAs: 1) Pick a state q to eliminate. 2) For every pair of remaining states (s, t) , replace the label from s to t by the regular expression

$$R_{s,t} \mid (R_{s,q} R_{q,q}^* R_{q,t})$$

Normally, from q_{big} to remove (the easiest one with less connections), only check the state (come to q , leave q) not $\emptyset$, others keep same

Solve regular-expression problem:

Convert R_i into a DFA M_i , then solve by DFA.

CFG: (V, Σ , R, S)

- **Variables** S, T (non-terminals)

- **Terminals** a, b, c (input characters) - **Rules** (row)

- **Start** variable S , $\rightarrow$ as **produces** or **rewrites** to

$S \Rightarrow T \Rightarrow c$ called a derivation.

The string of terminals is derived.

Palindromes: $S \rightarrow 0 \mid 1 \mid \epsilon \mid 0S0 \mid 1S1$

Same number: $S \rightarrow \epsilon \mid 0S1S \mid 1S0S$

Balanced (): $S \rightarrow (S)S \mid \epsilon$ (unambiguous)

(Parentheses) or $S \rightarrow (S) \mid SS \mid \epsilon$ (ambiguous)

Rightmost derivations:

if at each step it replaces the rightmost symbol.

Ambiguous grammars: A string is ambiguous if it has ≥ 2 parse trees in G or has ≥ 2 rightmost/leftmost derivations. The grammar G is ambiguous if it generates at least one ambiguous string.

The grammar itself is **unambiguous** because at every step of the derivation just there is just one possible rule to fire.

The union, concatenation, star closure of two CFL is also context-free. (star: add $S_0 \rightarrow SS_0 \mid \epsilon$)

• $\{ww : w \in \{a, b\}^*\}$ is not context-free

a) why your grammar only generates strings in

L: $L(G) \subseteq L$ If a string s is generated, then either the derivation starts $[S \rightarrow]$, in which case $[s =]$ where u is $[target]$ (by induction on the length of the derivation), and so s is $[target]$, or it starts with $[S \rightarrow]$ (and the reason is similar), and so is s .

b) why your grammar generates all strings in L: $L \subseteq L(G)$

Every $[target]$ string is generated: if u is $[target]$, it is of the form $[u =]$ where s, t are $[target]$. Then (by induction on the length of the string), each of s, t is generated, and then so is u . e.g., if $[u =]$ then start with $[S \rightarrow]$ and proceed with a derivation of s

Chomsky Normal Form (CNF):

Every rule is in: 1. $A \rightarrow BC$ (BC is not S) 2. $A \rightarrow a$ 3. **Only permit $S \rightarrow \epsilon$**

Parse trees of a grammar in CNF are binary trees

If G is in CNF then every non-empty string $w \in \text{Lang}(G)$ can be derived in $2|w| - 1$ steps. ($A \rightarrow a$ exactly $|w|$, $A \rightarrow AB$ at least $|w| - 1$)

How many (rightmost) derivations with N steps are there? In general, at least 2^N , each step we may have two or more possible rules to fire for the variable

CFG to CNF:

1) Remove epsilon rules: 1. Find the 'nullable' variables 2. Remove $A \rightarrow \epsilon$ 3. If $B \rightarrow uAv$ is in R and A is nullable, then add the rule $B \rightarrow uv$ to R . 4. Repeat Step 3. 5. Add $S \rightarrow \epsilon$ to R if $S \Rightarrow^+ \epsilon$

2) Remove unit rules: 1. Find all pairs A, B such that $A \Rightarrow^+ B$ 2. Remove all unit rules. 3. If $A \Rightarrow^+ B$ and $B \rightarrow u$ is in R then add $A \rightarrow u$ to R 4. Repeat Step 3.

1. **START:** Add a new start variable S and the rule $S \rightarrow T$ where T was the old start variable.) 2. **TERM:** Replace every terminal a on the RHS of a rule by the new variable N_a and add $N_a \rightarrow a$. 3. **BIN:** Eliminate rules with more than two variables 4. **EPSILON:** Eliminate epsilon rules (except $S \rightarrow \epsilon$) 5. **UNIT:** Eliminate unit rules

Eg: $T \rightarrow aTb \mid \epsilon$ 2. $S \rightarrow T$ 3. $S \rightarrow T$
 $T \rightarrow aTb \mid \epsilon$ $T \rightarrow ATB \mid \epsilon$ $T \rightarrow AX \mid \epsilon$
 1. $S \rightarrow T$ $A \rightarrow a$ $X \rightarrow TB$
 $T \rightarrow aTb \mid \epsilon$ $B \rightarrow b$ $A \rightarrow a$ $B \rightarrow b$

4. $S \rightarrow T \mid \epsilon$ 5. $S \rightarrow AX \mid \epsilon$
 $T \rightarrow AX$ $T \rightarrow AX$
 $X \rightarrow TB \mid B$ $X \rightarrow TB \mid b$
 $A \rightarrow a$ $A \rightarrow a$ $B \rightarrow b$
 $B \rightarrow b$

CYK algorithm (Cocke-Younger-Kasami)

To solve the membership problem for CFG $O(n^3)$

(0,2)		
(0,1)	(1,2)	
(0,0)	(1,1)	(2,2)

Eg: To solve (0,2) combine every Variables in (0,0) and (1,2); (0,1) and (2,2)

whether or not $X \Rightarrow^+ \epsilon$: 1. Let Z consist of all variables A such that $A \rightarrow \epsilon$ is a rule of R 2. Repeat

until Z no longer changes: if $A \rightarrow \alpha$ is a rule of R where α a non-empty sequence of variables, and if every variable in α is in Z , then put A in Z .

whether or not $X \Rightarrow^+ Y$: 1. First replace G by an equivalent grammar with no epsilon transitions. 2. Let $R' = R$. 3. Repeat until R' no longer changes: If $A \rightarrow C$ and $C \rightarrow B$ are in R' then add $A \rightarrow B$ to R' . 4. If $X \rightarrow Y$ in R' return Yes, else return No.

Turing Machines:

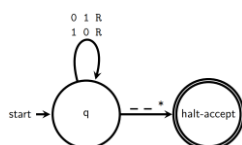
A “general purpose” programming language means expressively equivalent to a TM. A “computable” problem means implementable by a TM.

State: initial, **accept and reject**(halt); bi-infinite **tape**, made of **cells** (symbol: blank or character): a single pointer (head) on the tape; **transition rules; Input Alphabet Σ :** a subset of the Tape Alphabet, excluding blanks; **tape alphabet Γ ;** An **Input String** of the Input Alphabet is written on a tape with infinitely many blanks on each side. We call this tape an **Input Tape**

Morphett Notation (Formal description):
 $\langle \text{current state} \rangle \langle \text{current cell} \rangle \langle \text{new cell} \rangle \langle \text{direction} \rangle$
 $\langle \text{new state} \rangle (q \ 0 \ 1 \ R \ p)$

*****: $\langle \text{direction} \rangle$ for stay, $\langle \text{current cell} \rangle$ for any, $\langle \text{new cell} \rangle$ for no change

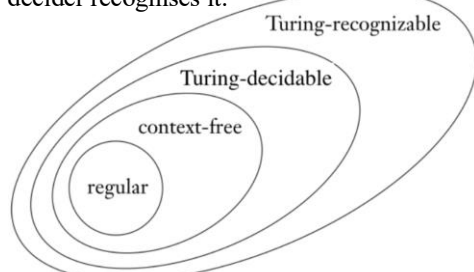
Visualising with State Diagrams:



Implementation description: English description of the way the Turing Machine moves its head.

The language **recognised** by TM M (or language of M) is the set of input strings M accepts. A language is **Turing-recognisable** (recognisable) if a TM recognises it.

A **decider** is a TM that halts on all inputs. A language is **Turing-decidable** (decidable) if a decider recognises it.



Using **multitracked** tapes: $\Sigma = \{0, 1\} \times \{m, b\}$, i.e., pairs (x, y)

free tape space space needed

Python programs are expressively equivalent to Turing Machines.

We say two Turing Machines are equivalent if they recognise the same language.

The Church-Turing Thesis: for every algorithm A , its input/output behaviour can be simulated by a Turing Machine M_A . Typically, an algorithm is defined as a procedure P (aka effective method)

satisfying properties such as: 1. P is set out in terms of a finite number of instructions. 2. P will produce the desired result in a finite number of steps. 3. P can (in practice or in principle) be carried out by a human being unaided by any machinery except paper and pencil. 4. P demands no insight, intuition, or ingenuity, on the part of the human being carrying out the method. $\Rightarrow$ **The acceptance problem for DFA and CFG are Turing-decidable.**

Universal Turing Machines (UTMs): NOT a decider
 $\{ \text{Source}_M; s \mid M \text{ is a TM and } s \text{ is an input that } M \text{ accepts} \}$

Ordinary TMs are expressively equivalent to Binary TMs. We can also construct a UTM in another expressively equivalent variant of TMs called **Multitape TM**. Multitape TMs are effectively TMs with multiple tapes and multiple, independently moving heads.

$x\#y$ to $x\#_y$: 1. Go to the end of y 2. For each character of y going leftwards: Turn the last character of y blank and write that character to the blank 1 cell to the right. 3. Halt accept once we reach the $\#$ character.

$L_{\text{diag}} = \{ \text{Source}_M : M \text{ is a TM that does not accept } \text{Source}_M \}$ is not recognizable Show $\text{Lang}(B) \neq L_{\text{diag}}$.

1. If $\text{Source}_B \in L_{\text{diag}}$, then B does not accept Source_B , so $\text{Source}_B \notin \text{Lang}(B)$. 2. If $\text{Source}_B \notin L_{\text{diag}}$ then B does accept Source_B , so $\text{Source}_B \in \text{Lang}(B)$

$L_{\text{TM-accept}} = \{ \text{Source}_M; w \mid M \text{ is a TM that accepts } w \}$ is undecidable and recognisable.

Contradiction:

The Halting Problem (undecidable, recognisable):
 $L_{\text{TM-halt}} = \{ \text{Source}_M; w \mid M \text{ is a TM that halts on } w \}$

Closure Properties: 1. **Union/Intersection**
 decidable/recognizable is decidable/recognisable. 2. If a language is decidable then so is its complement. 3. A language is decidable exactly when it and its complement are recognisable.

If a language is recognizable, we can't know its complement is recognisable.

$L_{\text{TM-not-accept}} = \{ \text{Source}_M; w \mid M \text{ is a TM that does not accept } w \}$ is undecidable.

Propositional Logic:

Syntax: conjunction($\wedge$); disjunction($\vee$); implication/conditional($\rightarrow$); atoms(p, q); formulas(F, G) [**recursive process:** 1. Every atom is formula, If F, G formula, 2. $\neg F$, 3. $(F \vee G)$, $(F \wedge G)$]; propositional constants: top/verum(T)[true]; bottom/falsum(L)[false]

abbreviations: drop outermost parentheses and repeated associative operations

A formula which occurs in another formula is called a **subformula**.

1. If G is an atom, then $\text{SubForms}(G) = \{G\}$. 2. If G is a formula of the form $\neg F$, then $\text{SubForms}(G) = \{G\} \cup \text{SubForms}(F)$. 3. If G is a formula of the form $(E \vee F)$ or $(E \wedge F)$ then $\text{SubForms}(G) = \{G\} \cup \text{SubForms}(E) \cup \text{SubForms}(F)$. Same as $\rightarrow, \leftrightarrow, \top, \perp$

Semantics: how you derive the value of a formula based on the values of its atomic subformulas

The elements of the set $\{0, 1\}$ are called **truth values**. 1 as “true”, and 0 as “false”

An **assignment** is a function α from atoms to truth values. **$\text{tv}(F, \alpha)$** , $\text{tv}(F, \alpha) = \alpha(F)$

$\alpha(F) = 1$: α makes F true / α satisfies/models F also written $\alpha \models F$ (double-turnstile)

A formula F is **valid** if every assignment satisfies F . (truth-table all 1/true) eg: $p \rightarrow (q \rightarrow p)$

A formula F is **satisfiable** if at least one assignment satisfies F . (truth-table has at least one 1/true)

Satisfiability problem: truth-table test (exponential time in the size of F)

use code that solves the satisfiability problem to solve the validity problem: pass $\neg F$ to the code

Equivalences:

(Idempotent Laws)	$F \equiv (F \wedge F)$ $F \equiv (F \vee F)$
(Commutative Laws)	$(F \wedge G) \equiv (G \wedge F)$ $(F \vee G) \equiv (G \vee F)$
(Associative Laws)	$(F \wedge (G \wedge H)) \equiv ((F \wedge G) \wedge H)$ $(F \vee (G \vee H)) \equiv ((F \vee G) \vee H)$
(Absorption Laws)	$(F \wedge (F \vee G)) \equiv F$ $(F \vee (F \wedge G)) \equiv F$
(Distributive Laws)	$(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H))$ $(F \vee (G \wedge H)) \equiv ((F \vee G) \wedge (F \vee H))$
(de Morgan's Laws)	$\neg(F \wedge G) \equiv (\neg F \vee \neg G)$ $\neg(F \vee G) \equiv (\neg F \wedge \neg G)$
(Double Negation Law)	$\neg\neg F \equiv F$
(Validity Law)	$(F \vee T) \equiv T$ $(F \wedge T) \equiv F$
(Unsatisfiability Law)	$(F \vee \perp) \equiv F$ $(F \wedge \perp) \equiv \perp$
(Constant Laws)	$T \equiv (F \vee \neg F)$ $\perp \equiv (F \wedge \neg F)$
(Negating constants Laws)	$\neg T \equiv \perp$ $\neg \perp \equiv T$
(Conditional Law)	$(F \rightarrow G) \equiv (\neg F \vee G)$
(Bi-conditional Law)	$(F \leftrightarrow G) \equiv ((F \rightarrow G) \wedge (G \rightarrow F))$

NAND ("not and"): $p \uparrow q$ defined as $\neg(p \wedge q)$
 $(p \uparrow q) \uparrow (p \uparrow q)$ is logically equivalent to $p \wedge q$

negation normal form (NNF): if negations only occur immediately in front of atoms.

conjunctive normal form (CNF): $(p \vee \neg q) \wedge (q \vee r)$
 -negate, then negate again; Distributive Law

Every F there is an equivalent formula in NNF/CNF.

Disjunctive Normal Form (DNF): $(p \wedge \neg q) \vee (q \wedge r)$

Same value: $(x \wedge y) \vee (\neg x \wedge \neg y) \equiv (\neg x \vee y) \wedge (x \vee \neg y)$

Different value: $(x \wedge \neg y) \vee (\neg x \wedge y)$

How to find constraint: duality trick: negate, write as DNF, and then negate that formula.

Logical consequence: $E_1, \dots, E_k \models F$

In case $k = 0$ we write $\models F$, which means that every assignment makes F true, i.e., that F is valid.

Predicate logic (First-order logic):

A **predicate** (of arity k) over domain D is a subset of D^k .
 - happy $\subseteq H$ is a unary predicate ($k = 1$).
 - loves $\subseteq H \times H$ is a binary predicate ($k = 2$).

Syntax: 1. A **term** is a variable or a constant symbol
 2. An **atomic formula** has the form $P(t_1, \dots, t_k)$ where P is a k -ary predicate symbol and $t_1, \dots, t_k$ are terms.

Quantifiers: existential $\exists$: $\exists x F$; universal $\forall$: $\forall x F$

"All A s are B s" translates as $\forall x(A(x) \rightarrow B(x))$
 "Some A s are B s" translates as $\exists x(A(x) \wedge B(x))$
 "No A s are B s" translates as $\forall x(A(x) \rightarrow \neg B(x))$
 "Some A s are not B s" translates as $\exists x(A(x) \wedge \neg B(x))$
 Usually: $\wedge$ goes with $\forall$; $\rightarrow$ goes with $\forall$

Negation: $\neg \forall x F \equiv \exists x \neg F$; $\neg \exists x F \equiv \forall x \neg F$ **Unification:**
 $(\forall x F \wedge \forall x G) \equiv \forall x (F \wedge G)$; $(\exists x F \vee \exists x G) \equiv \exists x (F \vee G)$

Transposition: $\forall x \forall y F \equiv \forall y \forall x F$; $\exists x \exists y F \equiv \exists y \exists x F$

Extraction: if $x \notin \text{Free}(G)$: $(\forall x F \wedge G) \equiv \forall x (F \wedge G)$;
 $(\forall x F \vee G) \equiv \forall x (F \vee G)$; $(\exists x F \wedge G) \equiv \exists x (F \wedge G)$;
 $(\exists x F \vee G) \equiv \exists x (F \vee G)$ [x not free or not occur in G]

Bound variables: if x occurs within a subformula of F of the form $\exists x G$ or $\forall x G$. Otherwise it is a **free**.

A formula without free variables is called a **sentence**.